Report - Version 9: Multi-Step Agent

Introduction

Version 9 introduces the Multi-Step Agent. In Version 8 the system could already route semantically and parse semantically, but it still executed only one action per request. Version 9 keeps the same tools, Tool Registry, LLM Router, and LLM Parser, and adds a simple form of sequential execution.

The Planner now creates an ordered list of steps, and the Agent executes those steps one at a time. The result of one step can be reused in the following step.

Goal of Version 9

The main goal of Version 9 is to introduce multi-step execution while keeping the architecture easy to understand.

- The Planner creates an ordered sequence of step requests.
- The Agent executes each step sequentially.
- The result of one step can be injected into the next step.
- The LLM Router and LLM Parser remain unchanged.
- The Memory Manager stores the full multi-step interaction.

What changes compared to Version 8

Version 8 already had an LLM Router and an LLM Parser, but the Planner returned a single plan and the Agent executed only one action. Version 9 changes the Planner so that it returns a list of plan items. It also changes the Agent so that it iterates over those items and keeps track of intermediate results.

New concept: multi-step planning

The Planner now splits requests containing sequential instructions separated by "then". Each sub-request is routed independently and converted into a step in the plan.

```
def split_request(request):
    return [
        step.strip(" ,.")
        for step in request.split(" then ")
        if step.strip()
    ]
```

For the example request below, the plan contains two ordered steps:

```
agent("Add 2 and 3, then multiply the result by 4.")
```

```
[
  {"request": "Add 2 and 3", "tool": "addition", ...},
  {"request": "multiply the result by 4", "tool": "multiplication", ...}
]
```

Passing the result to the next step

Version 9 adds a small helper that resolves references to the previous result. This keeps the implementation simple and didactic.

```
def resolve_step_request(request, previous_result):
    if previous_result is None:
        return request
    return request.replace("the result", str(previous_result))
```

After Step 1 computes 5, Step 2 receives the request "multiply 5 by 4" and the existing LLM Parser can extract the arguments normally.

Complete architecture in Version 9

The execution flow becomes:

```
Request -> Planner -> Step 1 (LLM Router -> LLM Parser -> Executor)
                   -> Step 2 (LLM Router -> LLM Parser -> Executor)
                   -> ...
                   -> Memory Manager -> Memory
                                        |
                                        v
                                     Output
```

Agent state in Version 9

The state now stores a list of plan items and a list of executed steps:

```
{
  "request": "Add 2 and 3, then multiply the result by 4.",
  "plan": [
    {"request": "Add 2 and 3", "tool": "addition", ...},
    {"request": "multiply the result by 4", "tool": "multiplication", ...}
  ],
  "steps": [
    {"request": "Add 2 and 3", "action": "addition", "arguments": {"a": 2,
        "b": 3}, "result": 5},
    {"request": "multiply the result by 4", "action": "multiplication",
        "arguments": {"a": 5, "b": 4}, "result": 20}
  ],
  "result": 20
}
```

Tests

Version 9 keeps the core single-step tests and adds a new multi-step test:

```
clear_memory()
tool_registry
agent("What is 2 + 3?")
agent("Hello, my name is luca.")
agent("What is the weather in Rome?")
agent("Could you calculate the product of 2 and 3?")
agent("Multiply three by four.")
agent("Add 2 and 3, then multiply the result by 4.")
get_memory()
get_last_interaction()
```

- The first five tests verify that Version 9 preserves the single-step behavior of Version 8.
- The new multi-step test verifies that the Planner creates two ordered steps.
- It also verifies that the result of the first step is passed to the second step and becomes part of the next parsed request.

- Memory now stores the entire multi-step interaction, including the full plan and the executed steps.

What Version 9 teaches

Version 9 teaches how an agent can move from one action to a small sequence of actions. The change remains intentionally simple: the planner splits on "then", the agent executes step by step, and the result of one step can be reused in the next one.

This version is the final implemented stage of the project. The system is now able to coordinate tools, store metadata in a Tool Registry, use a local model for routing and parsing, and execute small multi-step requests.

Limitations

- multi-step planning is intentionally simple and depends on the word "then";
- only references to "the result" are resolved explicitly;
- the small local model can still make routing or parsing mistakes;
- the Tool Registry remains in-memory and registration is manual;
- memory remains a simple list.

Conclusion

Version 9 is the natural conclusion of the notebook-based roadmap. It keeps the clean modular architecture developed in earlier versions and adds the final practical capability: multi-step execution.

The successful test "Add 2 and 3, then multiply the result by 4." clearly demonstrates the main benefit of the version. The agent can now plan a sequence, execute it step by step, pass intermediate results forward, and save the full interaction state.